Spring Boot Ders Notları

Computer Sales System projesi üzerinden, sorduğum sorular ve cevapları

Ders 0: Proje ve Spring Boot'a giriş

> **Soru: Bu proje ne yapıyor? Spring Boot nedir?**

Proje bir **REST API**: ekranı yok, istek alıp JSON cevap veren bir sunucu programı (backend). **Spring Boot**, Java ile sunucu yazmayı kolaylaştıran hazır altyapı (framework). Sunucu, veritabanı bağlantısı, JSON çevirme, SQL gibi işleri o hallediyor; biz sadece "ne yapılacağını" yazıyoruz.

Dosya / klasör	Görevi
`pom.xml`	Kullanılan kütüphanelerin listesi (Maven indirir)
`ComputerSalesSystemApplication.java`	Başlangıç noktası (`main` metodu)
`application.properties`	Ayarlar: veritabanı adresi, şifre vb.
`domain/`	Entity'ler = veritabanı tabloları
`repository/` , `service/` , `controller/`	Katmanlar (aşağıda)

Anotasyon (`@` ile başlayanlar): Spring'e "bu sınıf/alan özel, onunla şunu yap" diyen etiketler.

> **Soru: Bir istek uygulamanın içinde nasıl ilerliyor? (Katmanlı mimari)**

```
İstek → Controller → Service → Repository → Veritabanı
Cevap ← Controller ← Service ← Repository ← (veriler)
```

Katman	Restoranda	Görevi
Controller	Garson	İsteği alır, cevabı götürür
Service	Aşçı	Asıl iş, kurallar, hesaplama
Repository	Kiler	Veriyi getirir / kaydeder

Garson (Controller) kilere (Repository) doğrudan gitmez, aşçıya (Service) gider.

`JpaRepository` 'den türeyen repository'lerde hazır gelen metotlar: `findAll()` , `findById(id)` , `save(x)` , `deleteById(id)` .

Dependency Injection: `private final CategoryRepository repo;` nesnesini biz `new` ile oluşturmuyoruz, Spring veriyor.

Soru: Uygulama hangi portta çalışıyor? (3306 sandım)

```
Postman ──8080──▶ Spring Boot uygulaması ──3306──▶ MySQL
```

- **8080**: Bizim uygulamamız. Postman'de bunu yazıyoruz.
- **3306**: MySQL'in portu. Oraya uygulama bağlanıyor, biz değil.

Soru: `@Column(nullable = false, unique = true)` ne demek?

- `unique = true`: Bu değer tabloda benzersiz olmalı (TC kimlik no gibi).
- `nullable = false`: Bu alan boş bırakılamaz.

JSON ve entity ilişkileri

Soru: `OrderItem.order` nerede, sonsuz döngü neden oluyor?

`OrderItem` sınıfının içindeki `order` alanı. Karşı tarafı `Order` sınıfındaki `items` listesi. İki sınıf birbirini gösteriyor:

```
Order ──items──▶ OrderItem
  ▲                  │
  └──────order───────┘
```

Jackson (JSON'a çeviren kütüphane) Order → items → OrderItem → order → Order → ... diye sonsuza kadar gider ve `StackOverflowError` verir. Çözüm: döngüyü bir taraftan kesmek (`OrderItem.order` alanına `@JsonIgnore`).

Soru: `@JsonIgnore` ne işe yarıyor?

Jackson'a **"bu alanı JSON'a çevirirken atla"** der.

	Etkilenir mi?
API'nin döndürdüğü JSON	Evet, alan görünmez
Gelen JSON'u okuma (`@RequestBody`)	Evet, yok sayılır
Veritabanı kolonu	Hayır, değişmez
Java'da `getOrder()`	Hayır, normal çalışır

Kullanım yerleri: döngü kırmak, şifre gibi hassas alanı gizlemek, gereksiz veriyi göstermemek.
Dezavantajı: her yerde geçerli; büyük projelerde bunun yerine DTO kullanılır.

Ders 1: Uygulamayı çalıştırma, Postman, HTTP

Soru: H2 nedir, nasıl çalıştırıyoruz?

H2, uygulamanın içinde bellekte çalışan küçük bir veritabanı. Kurulum gerekmez; uygulama kapanınca veriler silinir.

- `pom.xml`'de H2'nin `scope`'u `runtime`: uygulama çalışırken de kullanılabilir (`test` = sadece testlerde).
- `application-h2.properties` bir **profil**: sadece "h2 profiliyle çalış" denince okunur ve ana ayarların üzerine yazar.

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=h2
```

H2 konsolu: `http://localhost:8080/h2-console` — JDBC URL `jdbc:h2:mem:computer_sales`, kullanıcı `sa`, şifre boş.

Soru: Bu `filter` nereye gidiyor?

```
@PostMapping("/filter")              ← ADRES (URL'nin parçası)
public List<Category> filter(...)    ← sadece METODUN ADI
```

Tam adres: `POST /api/categories/filter`. Yolculuk:

1. **Controller**: `@RequestBody` gelen JSON'u `RequestDto` nesnesine çevirir.
2. **Service**: `findByFilters` metodu `CategoryService` dosyasında görünmez; `implements IBaseFilterService` sayesinde oradan "miras" gelir (`default` metot, 10 servis ortak kullanır).
3. **FiltersSpecification**: filtreyi SQL `WHERE` koşuluna çevirir.
4. **Repository**: SQL'i çalıştırır, sonuç JSON olarak geri döner.

Soru: GET ile POST farkı ne?

	GET	POST
Amaç	Veri okumak	Veri göndermek (genelde yeni kayıt)
Veri nerede?	Adreste	Gövdede (body), JSON

Filter ucu okuma yaptığı halde POST kullanıyor, çünkü karmaşık filtre listesini gövdede göndermek daha kolay. Arama uçlarında yaygın bir istisna.

Soru: Postman'de 400 hatası aldım, hata mesajı nasıl okunur?

Uzun hatada sadece 3 alana bak: `status` (ne tür), `message` (ne oldu), `path` (nerede). `trace` kısmını şimdilik okumana gerek yok.

Kod	Anlamı	Kimin hatası?
2xx (200)	Başarılı	—
4xx (400, 404)	İstek hatalı	İsteği gönderenin (Postman)
5xx (500)	Sunucu hatası	Kodun (Java)

- `Required request body is missing` → gövde gönderilmemiş. Postman: **Body** → **raw** → **JSON** seç.
- `JSON parse error: Unrecognized token 'yakup'` → gövde geçerli JSON değil.
- `No static resource ...` (404) → o adreste bir uç yok (ya da kod derlenmemiş).

Soru: JSON nasıl yazılır?

- Nesne süslü parantez içinde: `{ ... }`
- Anahtarlar ve metinler **çift tırnak** içinde: `"name": "Laptop"`
- Sayılar tırnaksız: `1500`; liste köşeli parantez: `[1, 2]`
- En basit geçerli JSON: `{}`

`{"name": "yakup"}` gönderince hata gelmedi ama filtre de çalışmadı: `RequestDto`'da `name` alanı yok, Spring tanımadığı anahtarları **sessizce yok sayar**. "Hata almadım" her zaman "istediğim oldu" demek değil.

Soru: Olmayan bir kolonla (`isim`) filtre atarsam ne olur?

Cevap: `500` + `"Geçersiz kolon: isim"`. Mesaj doğru ama kod yanlış: hata istemcinin olduğu için 400 olmalı. Bunu ileride hata yönetimi (`@RestControllerAdvice`) dersinde düzelteceğiz.

Ders 2: İlk uç — tüm kategorileri listele

> **Soru:** **Bir Java metodu neyden oluşur?**

```
public  List<Category>  findAll  ()  {  return repo.findAll();  }
  ①          ②            ③      ④   ⑤           ⑥
```

1. `public` : herkes çağırabilir
2. Dönüş tipi: ne geri verecek
3. Metodun adı
4. Parantez: dışarıdan aldığı değerler (boşsa hiçbir şey almaz)
5. Süslü parantez: gövde
6. `return ...;` sonucu geri ver (noktalı virgül unutma)

> **Soru:** **Metodun adı `findAll`, içinde de `repo.findAll()` var. Karışmaz mı?**

Karışmaz. `repo.` "repo nesnesinin findAll metodu" demek. Nokta, **nesne ile metodu bağlar**: `service.findAll()`, `repo.findAll()`. Nokta olmadan yazılsa metot kendi kendini çağırırdı.

> **Soru:** **Controller'da ne yazdık, adres neden `/getAll` değil?**

```
@GetMapping
public List<Category> findAll() {
    return service.findAll();
}
```

Sınıftaki `@RequestMapping("/api/categories")` ile metottaki adres birleşir:

```
"/api/categories" + @GetMapping              = GET /api/categories
"/api/categories" + @GetMapping("/categories") = /api/categories/categories  (yanlış)
```

REST kuralı: adreste fiil olmaz. İşi HTTP metodu söyler.

Yanlış	Doğru
`GET /api/categories/getAll`	`GET /api/categories`
`POST /api/categories/create`	`POST /api/categories`
`DELETE /api/categories/delete/1`	`DELETE /api/categories/1`

Soru: Derleme hataları nasıl okunur?

`Dosya.java:[31,12]` = 31. satır, 12. karakter.

Hata	Anlamı / bendeki sebebi
`cannot find symbol: variable repo`	Bu isimde değişken yok (Controller'da `repo` değil `service` var)
`variable service is already defined`	Aynı değişken iki kez tanımlanmış
`reached end of file while parsing`	Dosya sonunda fazlalık/eksik (kaçak `ş` harfi; Türkçe klavyede `;` yerinde)
`illegal start of expression`	Java o noktada beklemediği bir şey gördü (metot içinde metot, yanlış yerde nokta)

Kural: Her zaman **ilk hatayı** düzelt, tekrar derle. Arkadan gelen hataların çoğu zincirleme hatadır, kendiliğinden kaybolur.

Java büyük/küçük harfe duyarlı: `@pathVariable` ≠ `@PathVariable`. Anotasyonlar büyük harfle başlar.

Soru: Uygulamayı yeniden başlatmadım ama yeni kod çalıştı, neden?

`spring-boot-devtools` derlenmiş dosyaları izler; kod derlenince uygulamayı kendisi yeniden başlatır. Terminalde yeni bir `Started ...` satırı çıkmadıysa Ctrl + C ile durdurup elle başlat. Not: H2 bellekte olduğu için yeniden başlayınca veriler silinir.

Soru: Commit nedir, GitHub Desktop'ta nasıl atılır?

Commit, projenin o anki halinin fotoğrafı; bir şey bozulursa geri dönülebilir. GitHub Desktop: **Changes** sekmesinde değişikliklere bak → sol altta **Summary** yaz → **Commit to <dal>**. **Publish branch** dalı GitHub'a gönderir (`main` değişmez).

Ders 3: Tek kategoriyi getirmek

Soru: `@PathVariable` ne işe yarıyor, neden koyduk?

Adresin içindeki değeri alıp metodun değişkenine koyar.

```
@GetMapping("/{id}")                          ← adreste "burada değer var, adı id"
public Long findById(@PathVariable Long id) {  ← "o değeri al, id değişkenine koy"
    return id;
}

/api/categories/5   →   id = 5
```

Koymasaydık Spring değeri **nereden** alacağını bilemezdi.

Anotasyon	Değeri nereden alır?
`@RequestBody`	İsteğin gövdesinden (Postman Body)
`@PathVariable`	Adresin içinden

`{id}` içindeki isim ile parametre adı (`id`) aynı olmalı. `{ }` olmazsa "id" sabit bir yazı sanılır.

`/api/categories/abc` → **400**: parametre `Long` olduğu için Spring "abc"yi sayıya çeviremez ve hatayı isteği gönderene yükler.

Soru: `Optional` nedir, olmayan kategori neden 200 + null dönüyordu?

`repo.findById(id)` doğrudan kategori değil bir **kutu** (`Optional<Category>`) döndürür: kategori varsa dolu, yoksa boş. Kutuyu açmadan döndürünce boş kutu `200 OK + null` olarak gidiyor; bu yanıltıcı, doğrusu **404 Not Found**.

```
public Category findById(Long id) {
    return repo.findById(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
"Kategori bulunamadı: " + id));
}
```

- `orElseThrow` : kutu doluysa içindekini verir, boşsa hata fırlatır.
- `() ->` : "kutu boşsa şunu çalıştır" kalıbı.
- `ResponseStatusException` + `HttpStatus.NOT_FOUND` : Spring bu hatayı 404 olarak döndürür.
- Service artık `Category` döndürdüğü için Controller'ın dönüş tipi de `Category` olmalı; yoksa `incompatible types` derleme hatası çıkar.

Araçlar

> **Soru: VS Code eskiden kodu otomatik tamamlıyordu, artık yapmıyor. Neden?**

Java eklentisi bu projede sadece **hafif modda** (Lightweight / syntax server) açılmıştı. Bu modda proje sınıfları ve Spring için tamamlama çalışmaz.

1. Sol altta **Restricted Mode** yazıyorsa tıkla → **Trust** (klasöre güven).
2. Alt durum çubuğunda Java simgesi `{ }` / "Lightweight Mode" görünüyorsa tıkla, ya da Cmd+Shift+P → **Java: Switch to Standard Mode**.
3. Hâlâ olmuyorsa Cmd+Shift+P → **Java: Clean Java Language Server Workspace** → Restart.

Proje yüklenirken sağ altta "Importing Maven project" yazar; bitince tamamlama gelir. Klasörü `GitHub/computer-sales-system-spring-boot` olarak (proje kökünden) açmak gerekir.